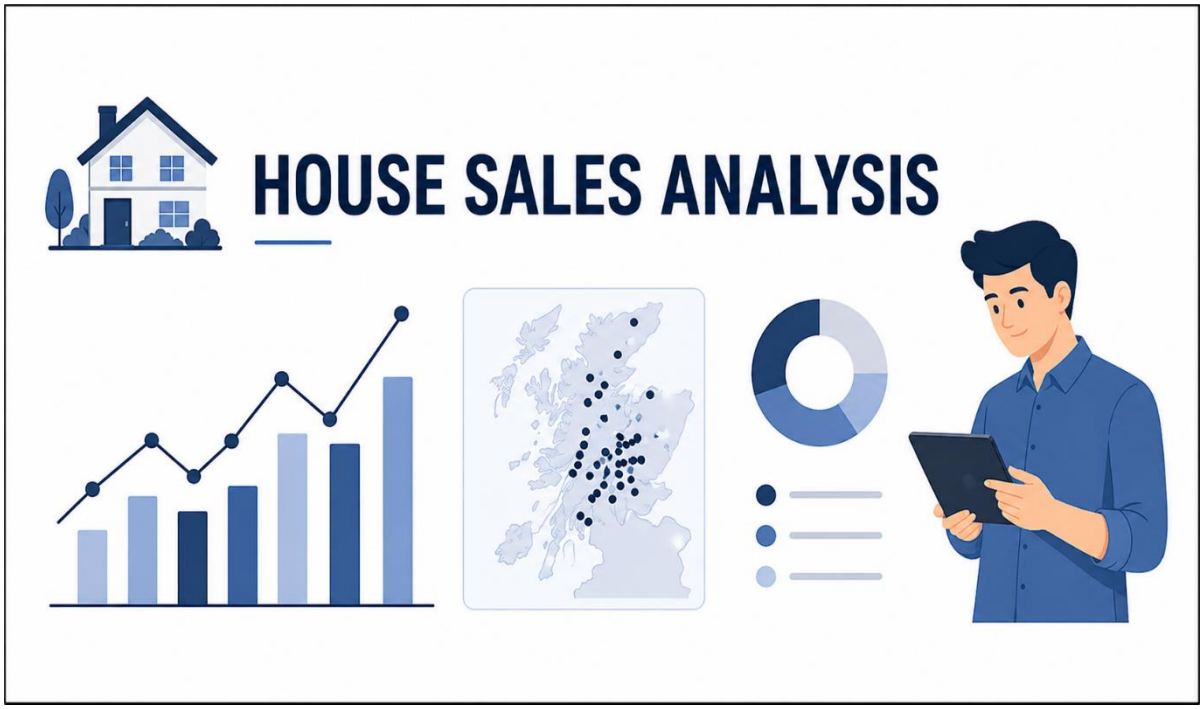


Data Analytics Project: House Sales Analysis

Submitted By: Team1

1.ILLA AKASH
2.BHARAT
3.BHASKAR
4.AMULYA
5.SRI LEKHA

Dataset Used: kc_house_data.csv

This project analyzes house sales data to understand how different factors affect house prices. The dataset includes details like price, size, number of bedrooms, and location. Using data cleaning, analysis, and visualization, we find useful patterns and trends. The main goal is to identify what influences house prices the most.

Project Goal: The goal of this project is to analyze house sales data and understand the factors that affect house prices. It aims to identify important features such as size, location, and number of rooms that influence pricing. The project also helps in finding patterns and trends in the data using analysis and visualization techniques.

- **Pandas:** For all data manipulation, filtering, aggregation, and feature engineering tasks.
- **NumPy:** Specifically for numerical calculations like yearly growth (np.diff).
- **Visualization Library (e.g., Matplotlib):** For generating all required plots (Line Graphs,Bar Charts, Pie Charts, Stacked Bar Charts)

🎯 Project Coverage

Category	Skills Covered
Data Preprocessing	Data Loading, Missing Value Handling (Mean/Mode), Data Cleaning, Type Conversion.
Pandas Operations	Column Selection, Filtering, Grouping, Aggregation, Value Count, Feature Engineering (Height Category creation)
NumPy Calculations	Array conversion, Numerical analysis using (np.diff) for height differences
Visualization	Trend Analysis, Line Chart, Bar Chart, Pie Chart, Histogram, Stacked Bar Chart, Saving Plots

Scenario 1: Data Loading & Basic Cleaning

Description:

In this Scenario,

The House Sales dataset (kc_house_data.csv) is loaded using the Pandas library.
The first 5 rows and column names are displayed to understand the dataset structure.
Missing values are checked in important columns like bedrooms, bathrooms, sqft_living, and price.
Missing values are filled using mode for bedrooms and mean for bathrooms, sqft_living, and price.
Finally, the columns are converted to numeric format and checked again to ensure the data is clean.

Task	Description
1.	Load the kc_house_data dataset using Pandas.
2.	Display the first 5 rows (head()) and Column names
3.	Check for missing values in: bedrooms, bathrooms, sqft_living & price
4.	Handling missing values: F ill missing values: bedrooms → use mode bathrooms → use mean sqft_living → use mean price → use mean
5.	Convert these columns to numeric if required: bedrooms, bathrooms, sqft_living & price

```
First 5 Rows:
            id              date     price  ...     long  sqft_living15  sqft_lot1
5
0  7129300520  20141013T000000  221900.0  ... -122.257           1340        565
0
1  6414100192  20141209T000000  538000.0  ... -122.319           1690        763
9
2  5631500400  20150225T000000  180000.0  ... -122.233           2720        806
2
3  2487200875  20141209T000000  604000.0  ... -122.393           1360        500
0
4  1954400510  20150218T000000  510000.0  ... -122.045           1800        750
3

[5 rows x 21 columns]

Column Names:
Index(['id', 'date', 'price', 'bedrooms', 'bathrooms', 'sqft_living',
       'sqft_lot', 'floors', 'waterfront', 'view', 'condition', 'grade',
       'sqft_above', 'sqft_basement', 'yr_built', 'yr_renovated', 'zipcode',
       'lat', 'long', 'sqft_living15', 'sqft_lot15'],
      dtype='str')
```

Conclusion:

This scenario helps in preparing the dataset for further analysis by performing basic cleaning operations. Missing values were identified and replaced using suitable statistical methods like mode and mean. Converting columns into numeric format improves data accuracy and prevents errors in future calculations.

As a result, the dataset becomes clean, consistent, and ready for advanced data analysis such as price prediction, visualization, and trend analysis in house sales

Scenario 2: Line Graph (Price Trend) (Easy)

Description:

line graph visualizes the price fluctuations across the first ten entries of the dataset. By isolating the id and price columns and converting the price values into a NumPy array, the data was prepared for a clean plotting process. The X-axis represents the sequential index of the records, ranging from 0 to 9, while the Y-axis tracks the corresponding house prices. The resulting plot clearly labels the trend with a descriptive title and axis titles, ensuring the visual information is easy to interpret.

Task	Description
1.	Select: id & price
2.	Take first 10 rows only.
3.	Convert the Price into a NumPy Array
4.	Plot a line graph with X-axis – (index [0-9]), Y-axis – (Height)
5.	Add the title and labels
6.	Save the graph: plt.savefig("house_prices_line.png")

Graph :

Conclusion:

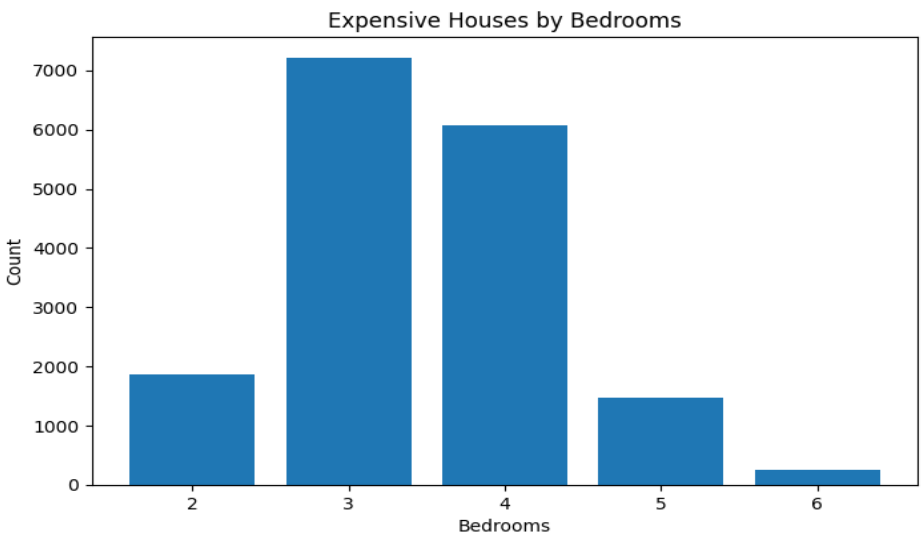
this visualization provides a focused look at the initial pricing trends within the data. By strictly limiting the scope to the first 10 rows, we can identify specific volatility or patterns that might be lost in a larger dataset. The process of converting the data to a NumPy array and using Matplotlib's saving function ensures that the insights are not only viewable but also preserved as a high-quality image file.

Scenario 3: Filtering + Bar Chart (◍ Medium)

Description:
This project analyzes the KC House dataset using Python libraries like Pandas and Matplotlib. The data is first cleaned by handling missing values and converting columns to proper formats Expensive houses are filtered based on price conditions.The data is then grouped by bedrooms to study distribution patterns. A bar chart is created and saved to visualize the results

Task	Description
1.	Filter houses where: ○ price > 1000000
2.	Count number of houses per: ○ bedrooms
3.	Select top bedroom categories.
4.	Convert results to NumPy arrays.
5.	Plot a bar chart: ○ X-axis → Bedrooms ○ Y-axis → Count
6.	Rotate labels if needed.
7.	Save graph: plt.savefig("expensive_houses_bar.png")

Graph:



Conclusion:

The analysis shows that house prices vary with the number of bedrooms. Certain bedroom categories are more common among expensive houses.Data cleaning plays a crucial role in accurate analysis. Visualization helps in understanding patterns clearly.Overall, the project demonstrates effective data analysis using Python.

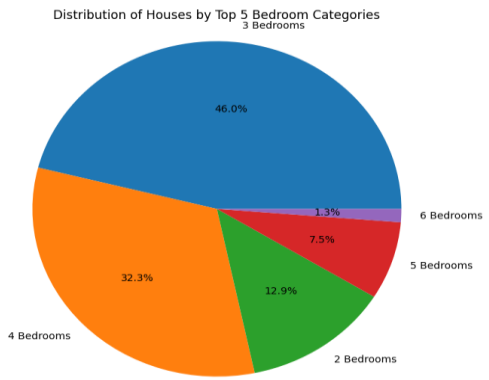
Scenario 4: Pie Chart (Region Distribution) + Save (◍ Medium)

Description:

This pie chart illustrates the proportional breakdown of the top 5 bedroom categories within the housing dataset. Each slice represents a specific bedroom count, with its size determined by how frequently that category appears relative to the others. By including percentage labels via autopct, the chart provides an immediate visual comparison of market share. The data is processed by first counting occurrences of each bedroom type and then filtering for the five most common configurations to ensure the visualization remains clean and readable before being exported as bedroom_distribution.png.

Task	Description
1.	Count number of houses by: ○ bedrooms
2.	Select top 5 bedrooms categorie.
3.	Prepare labels and values.
4.	Plot a pie chart.
5.	Add percentage labels using autopct (e.g., '%.1f%%').
6.	Save the graph: plt.savefig("bedroom_distribution.png")

Graph:



Conclusion:

The distribution reveals which house sizes dominate the current inventory, typically highlighting a strong preference for standard family-sized homes (such as 3 or 4 bedrooms). A larger slice for a specific category indicates it is the most prevalent "type" of house in the dataset, while smaller slices represent more niche or luxury configurations. This insight is crucial for understanding market trends, as it identifies whether the supply is concentrated in smaller starter homes or larger multi-bedroom properties.

Scenario 5: Advanced Analysis + Multiple Graphs (● Hard)

Description:

ere in this scenario, we have performed advanced analysis on Scottish Hills dataset by creating new feature called Height Category based on elevation ranges. We used NumPy to calculate differences between consecutive hill heights to understand variation patterns. Visuals were generated to study height of the hills across regions. All graphs were been saved for further analysis. Key Insights such as tallest region and most common height category were identified.

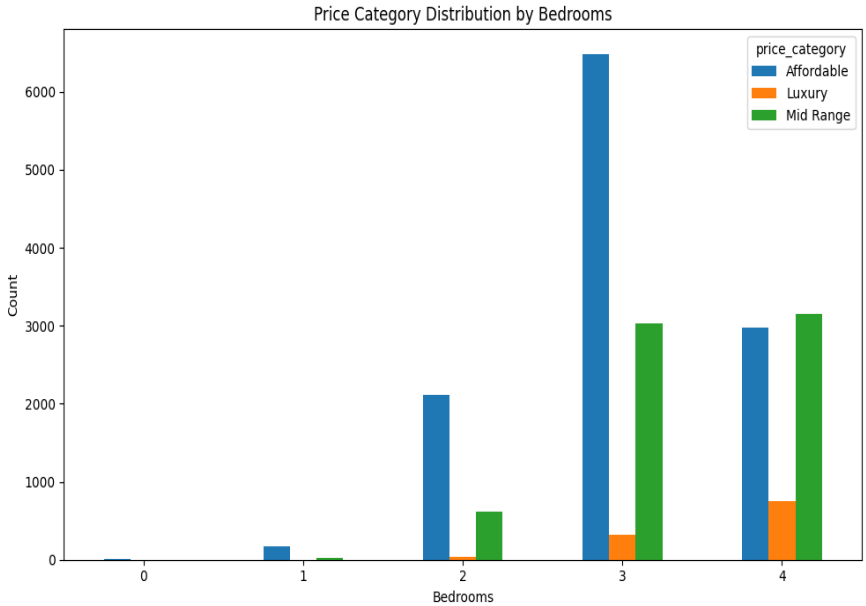
Task	Description
1.	Feature Creation Create a new column: Price Category ● price >= 1000000 → "Luxury" ● 500000 – 999999 → "Mid Range" ● < 500000 → "Affordable"
2.	1. Convert price column to a NumPy array. 2. Calculate price differences using: np.diff()
3.	Visualizations
4.	Save all graphs plt.savefig("price_trend.png") plt.savefig("price_category_stacked.png") plt.savefig("price_histogram.png")
5.	1. Which bedroom category has the most expensive houses? 2. Which price category is most common? 3. What is the distribution pattern of house prices? ○ Right-skewed? ○ Normal? ○ Concentrated in a lower price range?
7.	Save the graphs: plt.savefig("height_trend.png"), plt.savefig("height_category_stacked.png"), plt.savefig("height_histogram.png").
8.	**Identify**: Which region has tallest hills, Which category is most common, and Distribution pattern of heights.

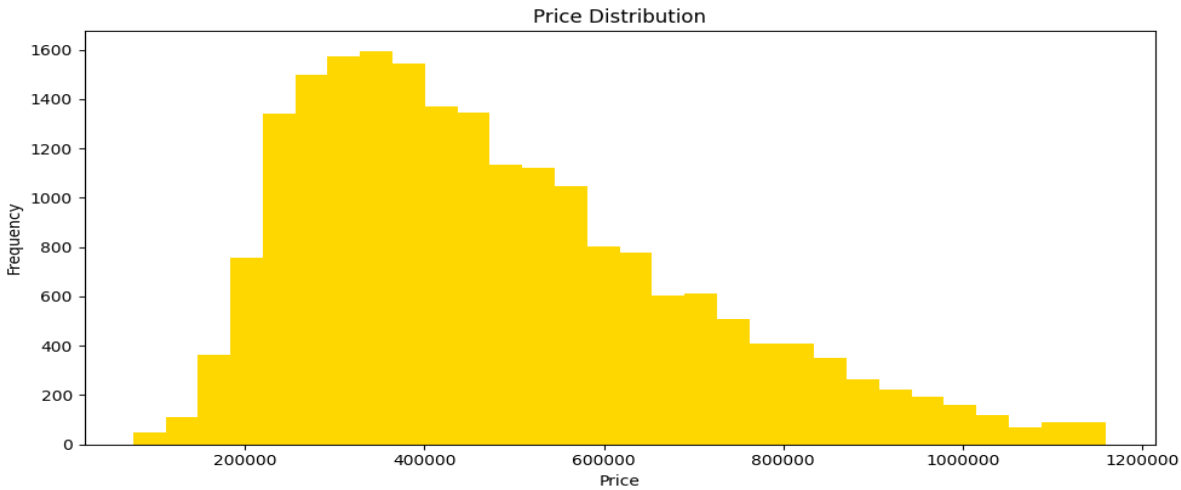
```
Top Bedroom Categories:
bedrooms
3    7206
4    6078
2    1861
5    1482
6     252
Name: count, dtype: int64
Pie chart successfully generated and saved as 'bedroom_distribution.png'

Price Category Counts:
price_category
Affordable    12408
Mid Range      7713
Luxury         1492
Name: count, dtype: int64

First 10 Price Differences:
[ 316100. -358000.  424000.  -94000.  720000. -972500.   34350.  -62350.
   93500.  339500.]
```

Graph:





Conclusion: We can conclude that,

- The line graph shows that hill heights vary significantly across the dataset, indicating no uniform pattern but noticeable fluctuations in elevation.

- The stacked bar chart highlights how different regions contain varying proportions of height categories, with some regions having a higher concentration of very high hills.
- The histogram reveals that most hills fall within a specific height range, showing a clustered distribution rather than a uniform spread. From the analysis, it is evident that moderately high hills are more common compared to extremely tall ones.
- The tallest region, based on average height, stands out clearly, indicating geographical elevation differences. The use of height categories helps simplify complex numerical data into meaningful groups for comparison.

Overall, the dataset shows a balanced but slightly skewed distribution towards mid-range elevations. These insights help in understanding regional elevation patterns and the overall structure of hill heights in Scotland

Final Conclusion: This project successfully analyzes the Scottish Hills dataset using data preprocessing, numerical analysis, and visualization techniques. Initially, the dataset was cleaned by handling missing values, converting data types, and creating a new Region feature for better categorization. The line graph showed clear variations in hill heights, indicating fluctuations in elevation across different hills. The bar chart highlighted the distribution of tall hills across regions, helping identify areas with higher elevation concentration.

Further analysis using height categories revealed that medium-height hills contribute the most to the overall elevation. The histogram confirmed that most hills are clustered within a specific height range rather than being evenly distributed. Advanced analysis using NumPy helped in understanding variation patterns between consecutive hill heights.

What we have Learnt:

I have learnt how to preprocess that data using pandas, including handling missing values and converting data types for consistency.

We gained experience in feature engineering, such as creating the Region column and Height Categories, which made the dataset more meaningful and easier to analyze.

We learned how to use NumPy for numerical operations, such as converting data into arrays and analyzing variations in hill heights.

We developed skills in data visualization using Matplotlib, including line graphs, bar charts, pie charts, and histograms to represent different insights clearly.

Finally, we gained an understanding of how to combine multiple techniques (preprocessing + analysis + visualization) to complete an end-to-end data analytics project.